



Unit 7 Quiz (Friday November 7) [Click for Study Guide]

Completion requirements

Due: Sunday, December 7, 2025, 10:20 AM

DSC 360 — Unit 7 Review Guide: LLM-Based Agents

Note: *This is now complete. I may add an essay question or two but that would only apply to your preparation for the second paper test.*

Scope and format. This quiz tests concepts from Unit 7, with primary emphasis on “What Is an Agent?” (supported by “Agents in Practice”). Questions will be a mix of multiple choice and short discussion, targeting conceptual understanding and precise terminology.

Review both slide decks and your class notes carefully—the slides don’t cover everything—and read the brief notes that accompany the demo code. While you won’t be asked for code specifics, you should be comfortable with the overall agent pattern and vocabulary: how the loop (perceive → decide → act → observe → adjust), sensors/actuators, safety checks, and KB-backed retrieval fit together conceptually.

Also rewatch the Week 1 video on agents; if you encounter a paywall, use the posted transcript.

What you should be ready to do.

- List the **task-environment specification** items (**Performance measure, Environment, Actuators, Sensors** — “PEAS”) and briefly **explain each**, applying them to a simple example such as the one from class
- Describe the **basic agent loop** for LLM-based systems
- Explain, at a high level, how an LLM agent can use a **knowledge base (KB)** and how that ties into **retrieval** (RAG) for grounding and safety

Terms to Know

- **Agent (classical AI).** A system that **perceives** its environment and **acts** upon it to achieve goals
- **Percept.** A single input the agent receives. Not to be confused with *precept*
- **Percept sequence.** The history of inputs so far

- **Environment.** The world (digital or physical) in which the agent operates
- **Sensors.** How the agent obtains information (APIs, files, databases, user prompts, tool outputs)
- **Actuators.** How the agent acts (tool calls, queries, messages, file writes, API requests)
- **Performance measure.** The criterion for success (e.g., correctness, safety, latency, clarity)
- **Rational agent (classical AI).** Chooses actions that are expected to best achieve the performance measure given what it has perceived and its computational limits
- **Basic agent loop.** Perceive → Decide → Act → Observe → Adjust (bounded iterations, not open-ended)
- **Goal specification.** A clear statement of what the agent should accomplish (used to guide decisions in the loop)
- **Policy/strategy.** The mapping from percepts (and state) to actions; in LLM agents, often realized via prompts plus simple control logic
- **Tool use/function calling.** Letting the agent take actions through approved tools (APIs, DB queries, file ops), with arguments the model proposes
- **Safety envelope.** Guardrails that constrain behavior (e.g., allow-list/block-list of tools and actions)
- **Validation.** Checks applied to model outputs or planned actions (schema checks, single-statement limits, read-only checks, argument validation, JSON schema/structured output)
- **Read-only policy.** Ensuring actions cannot change state when that is a requirement (e.g., analytics tasks)
- **Execution limits.** Timeouts, rate limits, and **bounded iteration** caps to keep behavior predictable
- **Grounding and retrieval.** Supplying task-relevant context from a KB to reduce hallucination and improve accuracy
- **Knowledge base (KB).** The information source(s) an agent relies on:
 - **Structured** (e.g., SQL schemas, table stats, tool manifests)
 - **Semi-structured** (YAML/JSON specs, config, metadata)
 - **Unstructured** (documents, pages, notes)
 - **Vector index** (embeddings for semantic retrieval)
- **RAG (Retrieval-Augmented Generation).** A pattern where the agent retrieves relevant KB chunks at decision time to ground its reasoning and actions
- **Transparency.** The property of being inspectable and understandable
- **Pre-flight refusal** (or pre-action refusal). Refusing unsafe or out-of-scope requests before any action is taken. Note: This is a special case of **safe refusal** conducted before any action is taken.

- **Execution trace (audit trail).** A structured record of prompts, tool calls, results, decisions, errors, and timestamps that enables transparency, debugging, and review

Core Concepts (concise reminders)

Agent vs. one-shot pipeline. A one-shot pipeline maps input→output once; an agent **loops**, using observations from the environment to refine or stop. Keep the loop small and deliberate

Classical AI framing for LLM agents. We analyze LLM systems using classical agent vocabulary (agent, environment, sensors/actuators, performance measure, rationality) because it clarifies **what** the system should optimize and **how** it interacts with the world

Task-environment specification (PEAS). Be ready to **list and explain** the four items and apply them to a small example (e.g., a read-only data assistant, a scheduling helper). Keep it focused: just the four items, one-line justifications, and how they relate to the task

KB and retrieval. An LLM agent's reliability improves when it consults an appropriate KB. Retrieval selects **just-in-time** context (schemas, definitions, examples, policy snippets), which the agent then uses to produce grounded, safer actions

Safety and validation (at a high level). Prefer **allow-listed** tools/actions, structured outputs (JSON with validation), **read-only** defaults for analytics tasks, single-statement limits for queries, and **timeouts/iteration caps** to control cost and failure modes

Example Questions (MCQ and short discussion)

- Define **agent** and **environment** in one or two sentences, using classical AI language
- Given a brief scenario, identify **sensors** and **actuators**
- What is a **performance measure** for a help-desk assistant? Provide two plausible criteria
- Describe the **basic agent loop** (one or two sentences) and explain why we bound iterations
- A system calls third-party APIs. Give two elements of a **safety envelope** that keep it predictable
- Explain how a **knowledge base** and **retrieval** help an LLM agent answer accurately
- In a task-environment specification, list and briefly **explain** the four items for a simple read-only data assistant

- What does **validation** mean in the context of LLM agents, and give one concrete example

Final notes

- Focus on **concepts and terminology**; you won't be asked to reproduce code
- Be prepared to **list, explain, and apply** the task-environment specification items to a simple example like the one shown in class
- Expect a blend of **MCQs** and **short discussion** prompts that test understanding of the **basic agent loop**, **safety/validation**, and **KB + retrieval** grounding